

Natural Language Processing (CS24311) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & LANGUAGE MODELS BANK

Natural Language Processing (CS24311)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024-25 Scheme)

 Complete Course Structure & NLP Architecture Matrix

Module	Core Syllabus Scope	Key Algorithms & Formulations
Module I	Morphology & Language Modeling	Regular Expressions, Porter Stemmer, Byte-Pair Encoding (BPE), N-grams, Perplexity, Laplace, Good-Turing & Kneser-Ney Smoothing
Module II	Part-of-Speech Tagging & HMM	Penn Treebank Tagset, Hidden Markov Models (HMM), Forward Algorithm, Viterbi Dynamic Programming Decoding
Module III	Syntactic Parsing & Grammars	Context-Free Grammars (CFG), Chomsky Normal Form (CNF), CYK Parsing Algorithm, Treebanks & Probabilistic CFG (PCFG)
Module IV	Semantics & Word Embeddings	WordNet Synsets, TF-IDF Vectorization, Pointwise Mutual Information (PMI), Word2Vec (Skip-Gram & CBOW), GloVe
Module V	Transformers, LLMs & Generation	Scaled Dot-Product Attention, Multi-Head Attention, Positional Encodings, BERT (Masked LM), GPT, BLEU Score & ROUGE

 Exam Preparation & High-Yield Strategy

This publication-grade master book consolidates all 5 modules with formal mathematical derivations, KaTeX-rendered language model smoothing formulas, Viterbi trellis dynamic programming traces, and model answers to BIT Mesra end-semester examination questions.

☰ Module I: Introduction to NLP & Preprocessing — Complete 12-Topic Tracker

Topic 1: Introduction to NLP (Scientific vs. Engineering Goals & Ambiguity)

Topic 3: Morphology (Morphemes, Free/Bound, Inflectional vs. Derivational)

Topic 5: Semantics (Literal Meaning & Semantic Role Labeling)

Topic 7: Tokenization (Word, Sentence, Subword BPE & Character)

Topic 9: Stop-Word Removal (Tradeoffs & Sentiment Preservations)

Topic 11: Regular Expressions (Pattern Matching & Information Extraction)

Topic 2: Real-World NLP Applications (Search, Healthcare, Finance, Chatbots)

Topic 4: Syntax (Grammatical Phrase Structures & Parse Trees)

Topic 6: Pragmatics (Contextual Meaning: S-S-P Paradigm)

Topic 8: Lemmatization (Base Lemma vs. Porter Stemming)

Topic 10: Normalization (Case Folding, Unicode & Contractions) [UPLOADED PYQ]

Topic 12: Language Models Overview (Sequence Probability Distributions)

Topic 1 & 2: Introduction to NLP & The Fundamental Ambiguity Challenge [UPLOADED PYQ]

Natural Language Processing (NLP) sits at the intersection of Artificial Intelligence and Computational Linguistics, pursuing two fundamental objectives:

Scientific Goal: Formulating computational models of human cognitive language understanding and production.

Engineering Goal: Developing robust, scalable software systems capable of processing human text and speech for practical real-world utility.

[UPLOADED PYQ — CS633] Why is Ambiguity the Core Challenge of NLP?

Natural language is inherently ambiguous across all linguistic tiers:

1. **Lexical Ambiguity:** A single word possesses multiple distinct meanings (e.g., "bank" → financial institution vs. river slope).
2. **Syntactic (Structural) Ambiguity:** A sentence can generate multiple valid parse trees (e.g., "I saw the man with the telescope" → who holds the telescope?).
3. **Semantic Ambiguity:** A sentence has multiple literal interpretations (e.g., "The chicken is ready to eat" → is the chicken hungry or cooked?).
4. **Pragmatic Ambiguity:** Meaning depends strictly on conversational context and real-world intent (e.g., "Can you pass the salt?" → an action request, not an inquiry about physical ability).

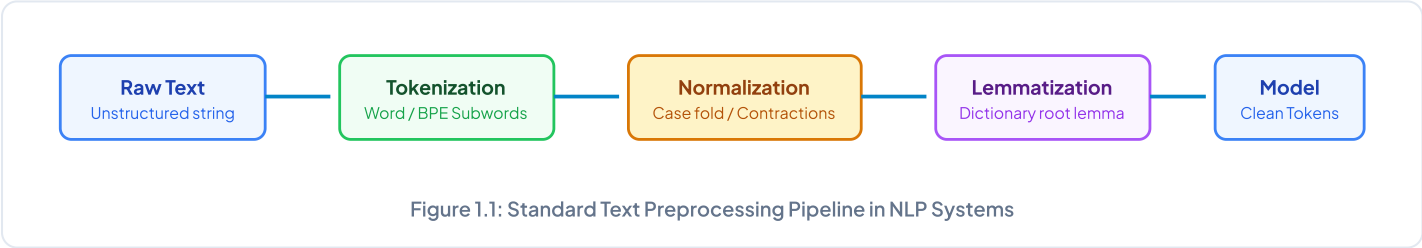
Topic 3 – 6: The 4 Core Levels of Linguistic Analysis

Linguistic Level	Core Scope & Analysis Mechanism	Representative Example
1. Morphology	Study of the internal structure of words and their minimal meaning-bearing units (Morphemes). • <i>Free</i>: Can stand alone ("happy"). • <i>Bound</i>: Prefixes/Suffixes ("un-", "-ness"). • <i>Inflectional</i>: Changes grammar form ("walk" → "walked"). • <i>Derivational</i>: Creates new words/parts of speech ("happy" → "unhappiness").	"unhappiness" = "un-" (prefix) + "happy" (root) + "-ness" (suffix).
2. Syntax	Formal grammatical rules governing how words combine into phrases and sentences (Context-Free Grammars).	$S \rightarrow NP VP$ (e.g., "The student [NP] solved the problem [VP]").
3. Semantics	The study of literal, context-independent linguistic meaning and predicate-argument relationships (Semantic Roles: Agent, Action, Patient).	"The dog chased the cat" → Agent = dog, Action = chase, Patient = cat.
4. Pragmatics	Meaning in real-world communicative context, discourse intent, and speaker implicature.	"It is freezing in here!" ⇒ Pragmatic command to close the window.

 **Memory Hook: S-S-P Paradigm**

Syntax (Structure) → Semantics (Sense / Meaning) → Pragmatics (Situation / Context)

Topic 7 – 10: Text Preprocessing & Normalization Pipeline [UPLOADED PYQ]



Feature	Stemming (e.g., Porter Stemmer)	Lemmatization (WordNet Lemmatizer) [UPLOADED PYQ]
Core Technique	Crude heuristic rule-based chopping of prefixes and suffixes.	Full morphological vocabulary and dictionary lookup analysis.
Output Validity	Often produces non-words (e.g., "running" → "run", "studies" → "studi").	Guaranteed valid dictionary base form (e.g., "better" → "good", "mice" → "mouse").
Context Awareness	Zero Part-of-Speech context awareness.	Uses POS tags to resolve ambiguity ("meeting" as Verb → "meet", as Noun → "meeting").

Topic 11 & 12: Regular Expressions & Language Models Overview

Language Model Formal Definition: A **Language Model (LM)** computes the joint probability distribution over any arbitrary sequence of n words $W = \langle w_1, w_2, \dots, w_n \rangle$:

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_1, w_2, \dots, w_{i-1})$$

M1 Active Recall & Exam Questions [UPLOADED PYQ]

Q1. [UPLOADED PYQ — CS633] Explain the 5 sequential phases of NLP with an end-to-end example. (10 Marks)

Given sentence: *"The intelligent student solved the complex problem."*

1. **Morphological Analysis:** Breaks tokens into morphemes ("solve" + "-ed", "complex").
2. **Syntactic Analysis (Parsing):** Builds hierarchical phrase parse tree ($S \rightarrow NP VP$).
3. **Semantic Analysis:** Assigns literal meaning (*Agent* = student, *Action* = solve, *Object* = problem).
4. **Discourse Integration:** Links pronouns and entities across surrounding sentences.
5. **Pragmatic Analysis:** Interprets the speaker's true intent within conversational context.

Module II: Language Models, POS Tagging & Parsing — Complete 12-Topic Tracker

Topic 13: N-Gram Language Models (Unigram, Bigram & Trigram Probabilities)

Topic 15: Rule-Based Part-of-Speech Tagging (Morphological Rules)

Topic 17: Context-Free Grammars (CFG Formal Definition & Parse Trees)

Topic 19: Probabilistic Parsing (PCFG, CNF & CKY Algorithm) [UPLOADED PYQ]

Topic 21: Precision Metric Formulation in NLP Evaluation

Topic 23: F1-Score Harmonic Mean Formulation

Topic 14: Smoothing & Perplexity (Laplace Add-1 & Kneser-Ney Backoff)

Topic 16: HMM POS Tagging & The Viterbi Algorithm [UPLOADED PYQ]

Topic 18: Recursive Descent Parsing (Top-Down Backtracking Search)

Topic 20: Dependency Parsing (Typed Dependency Trees & Arc-Standard)

Topic 22: Recall Metric Formulation in Information Extraction

Topic 24: Parsing Evaluation Metrics (UAS & LAS Attachment Scores)

Topic 13 & 14: N-Gram Models, Laplace Smoothing & Perplexity [UPLOADED PYQ]

1. Bigram Maximum Likelihood Estimation (MLE):

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

2. Laplace (Add-1) Smoothing Formula [UPLOADED PYQ]:

$$P_{\text{Laplace}}(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i) + 1}{\text{Count}(w_{i-1}) + |V|}$$

Where $|V|$ is the total vocabulary size. Guarantees zero-probability events never crash sentence likelihoods!

3. Model Perplexity Metric (PP):

$$PP(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}}$$

Interpretation: The average branching factor of the language model (Lower Perplexity $\implies$ Higher Predictive Accuracy).

[UPLOADED PYQ] Solved Problem: Bigram Probability with Laplace Smoothing

Given corpus counts: $\text{Count}(\text{"want"}, \text{"to"}) = 600$, $\text{Count}(\text{"want"}) = 1200$, Vocabulary $|V| = 5000$.

1. MLE Probability: $P_{\text{MLE}}(\text{"to"} | \text{"want"}) = \frac{600}{1200} = \mathbf{0.500}$.

2. Laplace Smoothed Probability: $P_{\text{Laplace}}(\text{"to"} | \text{"want"}) = \frac{600+1}{1200+5000} = \frac{601}{6200} \approx \mathbf{0.0969}$.

Topic 15 & 16: HMM POS Tagging & The Viterbi Algorithm [UPLOADED PYQ]

A **Hidden Markov Model (HMM)** models Part-of-Speech tagging as finding the most probable hidden sequence of POS tags $T = \langle t_1, \dots, t_n \rangle$ given observed words $W = \langle w_1, \dots, w_n \rangle$:

$$\hat{T} = \arg \max_T P(T | W) = \arg \max_T \prod_{i=1}^n P(t_i | t_{i-1}) \cdot P(w_i | t_i)$$

The Viterbi Dynamic Programming Algorithm [UPLOADED PYQ]

$$V_t(j) = \max_{i=1}^N [V_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$$

$$\text{Backpointer: } B_t(j) = \arg \max_{i=1}^N [V_{t-1}(i) \cdot a_{ij}]$$

Where $a_{ij} = P(t_j | t_i)$ is Tag Transition probability, and $b_j(o_t) = P(w_t | t_j)$ is Word Emission probability.

Topic 17 – 20: Syntactic Parsing (CFG, PCFG, CKY & Dependency Parsing) [UPLOADED PYQ]

Parsing Paradigm	Grammar & Structural Formulation	Key Advantages & Tradeoffs
1. Context-Free Grammar (CFG)	$G = (V, \Sigma, R, S)$ where productions have form $A \rightarrow \alpha$ ($A \in V$). Recursive descent expands rules top-down.	Precise; vulnerable to left-recursion infinite loops and combinatorial ambiguity explosions.
2. Probabilistic CFG (PCFG) [UPLOADED PYQ]	Assigns probability to each rule: $P(A \rightarrow \alpha)$, where $\sum_{\alpha} P(A \rightarrow \alpha) = 1$. Parse probability is product of rule probabilities.	Ranks multiple ambiguous parse trees and selects the mathematically most probable syntactic structure!
3. CKY Dynamic Programming [UPLOADED PYQ]	Requires grammar in Chomsky Normal Form (CNF) : $A \rightarrow BC$ or $A \rightarrow a$. Fills triangular table $O(n^3 \cdot R)$.	Guaranteed polynomial $O(n^3)$ parsing time for all ambiguous sentences!
4. Dependency Parsing	Connects words directly with labeled binary grammatical dependency links (e.g., nsubj, dobj, amod).	Directly captures predicate-argument relationships; excellent for free-word-order languages.

Topic 21 – 24: Parsing & Classification Evaluation Metrics

1. Classification / Parsing Core Metrics:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1\text{-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Unlabeled Attachment Score (UAS)**: Percentage of words assigned the correct syntactic head governor. - **Labeled Attachment Score (LAS)**: Percentage of words assigned both the correct head governor AND the correct dependency relation label.

M2 Active Recall & Exam Questions [UPLOADED PYQ]

Q1. [UPLOADED PYQ] Explain the working of HMM-based POS Tagging. Define Transition and Emission Probabilities. (10 Marks)

HMM treats POS tags as hidden Markov states generating observable surface words:

1. **Transition Probability** $P(t_i | t_{i-1})$: The probability of transitioning from tag t_{i-1} to tag t_i : $P(t_i | t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$.

2. **Emission Probability** $P(w_i | t_i)$: The probability that hidden tag t_i emits the observable word w_i : $P(w_i | t_i) = \frac{C(t_i, w_i)}{C(t_i)}$.

3. **Viterbi Algorithm**: Computes the global optimal sequence of tags in $O(T \cdot N^2)$ time using dynamic programming lattice trellis!

Module III: Semantics & Lexical Vector Resources — Complete 11-Topic Tracker

Topic 25: Lexical Semantics (Synonymy, Antonymy, Hyponymy, Meronymy)

Topic 27 & 28: WordNet Lexical Database (Synsets & Hypernym Hierarchies)

Topic 30: Cosine Similarity Formulation in Vector Spaces

Topic 32: TF-IDF Weighting Formulation & Numerical Analysis [UPLOADED PYQ]

Topic 34: GloVe (Global Vectors from Word-Word Co-Occurrence Counts)

Topic 26: Word Sense Disambiguation (Simplified Lesk & Supervised WSD)

Topic 29: Semantic Similarity (Shortest Path Length in WordNet)

Topic 31: Vector Space Models (Term-Document & Term-Term Matrices)

Topic 33: Word2Vec Embeddings (CBOW vs. Skip-Gram Architectures) [UPLOADED PYQ]

Topic 35: FastText (Subword Character N-Gram Vector Representations)

Topic 25 – 29: Lexical Semantics, WordNet & WSD Algorithms

Lexical Relation	Formal Definition	Representative Example
1. Synonymy	Different words expressing the identical or near-identical concept ($A \approx B$).	"large" $\leftrightarrow$ "big", "automobile" $\leftrightarrow$ "car".
2. Antonymy	Words expressing opposite semantic meanings along a spectrum ($A \leftrightarrow -A$).	"hot" $\leftrightarrow$ "cold", "increase" $\leftrightarrow$ "decrease".
3. Hyponymy / Hypernymy	IS-A class-subclass hierarchical relationship (A is a subtype of B).	"sparrow" is a hyponym of "bird"; "animal" is a hypernym of "dog".
4. Meronymy / Holonymy	PART-OF compositional relationship (A is a physical component of B).	"wheel" is a meronym of "car"; "car" is a holonym of "wheel".

The Simplified Lesk WSD Algorithm (Lesk, 1986)

Given target ambiguous word w in context sentence C :

1. Retrieve all dictionary definitions (glosses) and usage examples for each synset sense S_i of w in WordNet.
2. Compute the word token overlap score between the context sentence C and the gloss/examples of sense S_i :

$$\text{Overlap}(S_i, C) = |\text{Words}(S_i.\text{gloss}) \cap \text{Words}(C)|.$$
3. Select the sense $\hat{S} = \arg \max_{S_i} \text{Overlap}(S_i, C)$ with the highest word overlap count.

Topic 30 – 32: Vector Space Models & TF-IDF Weighting [UPLOADED PYQ]

1. Term Frequency-Inverse Document Frequency (TF-IDF) [UPLOADED PYQ]:

$$\text{TF}(t, d) = \frac{\text{Count}(t, d)}{\sum_{t' \in d} \text{Count}(t', d)}$$

$$\text{IDF}(t, D) = \log_{10} \left(\frac{N}{\text{DF}(t)} \right)$$

$$\text{TF-IDF}(\mathbf{t}, \mathbf{d}, \mathbf{D}) = \text{TF}(\mathbf{t}, \mathbf{d}) \times \text{IDF}(\mathbf{t}, \mathbf{D})$$

Where N is total documents in corpus, and $\text{DF}(t)$ is number of documents containing term t .

2. Vector Cosine Similarity Metric:

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \sqrt{\sum_{i=1}^d v_i^2}}$$

Topic 33 – 35: Neural Word Embeddings (Word2Vec, GloVe, FastText) [UPLOADED PYQ]

The Distributional Hypothesis (J.R. Firth, 1957)

"You shall know a word by the company it keeps!" Words appearing in similar contextual distributions acquire geometrically close spatial representations in d -dimensional embedding space.

Embedding Model	Training Objective & Operational Mechanism	Key Advantages & Tradeoffs
1. Word2Vec: CBOW [UPLOADED PYQ]	Continuous Bag-of-Words: Predicts current center target word w_t given surrounding context window words: $\arg \max \log P(w_t w_{t-c}, \dots, w_{t+c})$.	Fast to train; higher accuracy for frequent words; smooths over distributional noise.
2. Word2Vec: Skip-Gram [UPLOADED PYQ]	Predicts surrounding context words given center word w_t : $\arg \max \sum \log P(w_{t+j} w_t)$ using Negative Sampling.	Superb for capturing subtle semantic nuances of rare words; slower to train.
3. GloVe	Global Vectors (Stanford): Factorizes global log-co-occurrence matrix: $J = \sum f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$.	Combines global matrix factorization statistics with local context window advantages.
4. FastText (Meta AI)	Represents each word as a bag of Character n-Grams (e.g., 'nlp', 'lps').	Seamlessly computes embeddings for Out-of-Vocabulary (OOV) and morphologically complex words!

M3 Active Recall & Exam Questions [UPLOADED PYQ]

Q1. [UPLOADED PYQ] Explain the Continuous Bag of Words (CBOW) and Skip-Gram models of Word2Vec with neat network diagrams. (10 Marks)

- **CBOW Architecture:** Input layer takes $2C$ context word one-hot vectors $\rightarrow$ Projection layer averages context embeddings $\rightarrow$ Softmax output layer predicts the single probability distribution of center target word w_t .
- **Skip-Gram Architecture:** Input layer takes 1 center word one-hot vector $w_t \rightarrow$ Projection layer retrieves embedding $\rightarrow$ Output layer simultaneously predicts $2C$ context word probability distributions $\langle w_{t-C}, \dots, w_{t+C} \rangle$ using Negative Sampling ($\log \sigma(v'_{w_o}{}^T v_{w_I}) + \sum_{i=1}^k \mathbb{E}[\log \sigma(-v'_{w_i}{}^T v_{w_I})]$).

Module IV: Deep Learning & Neural Architectures in NLP — Complete 8-Topic Tracker

Topic 36: Convolutional Neural Networks (CNNs) for Text Classification

Topic 38: Long Short-Term Memory (LSTM) Gate Formulations & Memory Cells

Topic 40: Sequence-to-Sequence Encoder-Decoder with Cross-Attention

Topic 42: BERT (Bidirectional Encoder Representations from Transformers)

Topic 37: Recurrent Neural Networks (RNNs) & Vanishing Gradients

Topic 39: The Transformer Architecture (Scaled Dot-Product Self-Attention)

Topic 41: Transfer Learning in NLP (Pre-Training + Task Fine-Tuning)

Topic 43: GPT (Generative Pre-Trained Causal Autoregressive Decoders)

Topic 36 – 38: Sequential Neural Models (CNN, RNN, LSTM)

Neural Model	Mathematical Formulation & Architecture	Key Advantage & Failure Mode
1. CNN for NLP	1D convolution filters slide across embedding matrices capturing local n -gram phrase features: $c_i = f(\mathbf{w}^T \mathbf{x}_{i:i+h-1} + b) \rightarrow$ Max-Pooling over time.	Highly parallelizable; fails to model long-range sequential syntax dependencies.
2. Vanilla RNN	Recurrent hidden state: $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$. Unrolls across variable sentence lengths.	Models sequential history; severely suffers from Vanishing and Exploding Gradients over > 10 tokens.
3. LSTM (Hochreiter, 1997)	Regulates information flow using 3 Gates + Constant Error Carousel Cell State C_t : Forget Gate f_t , Input Gate i_t , Output Gate o_t .	Maintains long-range gradient highway over hundreds of tokens; sequential processing inhibits GPU scaling.

Complete LSTM Mathematical Gate Formulations:

$$\text{Forget Gate: } \mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

$$\text{Input Gate: } \mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \quad \tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

$$\text{Cell State Update: } \mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

$$\text{Output Gate: } \mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

Topic 39 & 40: The Transformer Architecture (Vaswani et al., 2017)

1. Scaled Dot-Product Attention Equation:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

Where $\mathbf{Q}$ (Query), $\mathbf{K}$ (Key), and $\mathbf{V}$ (Value) are linear projections of input embeddings. Division by $\sqrt{d_k}$ prevents gradient saturation in large dimensions!

2. Multi-Head Attention (MHA):

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad \text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

Topic 41 – 43: Pre-Trained Transformers: BERT vs. GPT

Feature	BERT (Devlin et al., Google)	GPT Series (Radford et al., OpenAI)
Core Architecture	Transformer Encoder Only (Bidirectional self-attention).	Transformer Decoder Only (Autoregressive causal masked attention).
Pretraining Objective	1. Masked LM (MLM: predicts 15% '[MASK]' tokens). 2. Next Sentence Prediction (NSP).	Standard Left-to-Right Causal Language Modeling: $\max \sum \log P(w_t \mid w_{<t})$
Optimal Tasks	NLU, Text Classification, Named Entity Recognition, Question Answering.	NLG, Creative Writing, Chatbots, Open-Domain Reasoning, Code Generation.

M4 Active Recall & Exam Questions

Q1. Explain the working of the Scaled Dot-Product Self-Attention mechanism in Transformers with a diagram. (10 Marks)

Self-attention calculates pairwise relational affinities between every pair of tokens in parallel:

- Linear Projections:** Input matrix $\mathbf{X}$ is multiplied by weight matrices $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$ to generate $\mathbf{Q}, \mathbf{K}, \mathbf{V}$.
- Score Calculation:** Computes dot-product similarity matrix $\mathbf{S} = \mathbf{Q}\mathbf{K}^T$, reflecting how much each token attends to all other tokens.
- Scaling & Normalization:** Divides by $\sqrt{d_k}$ to stabilize gradients and applies row-wise $\text{softmax}(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}})$.
- Weighted Aggregation:** Multiplies attention weights by value matrix $\mathbf{V}$ to output contextualized embeddings!

Module V: NLP Applications, Dialogue Systems & AI Ethics — Complete 10-Topic Tracker

- Topic 44:** Text Classification (Document Categorization & Naive Bayes)

Topic 46: Named Entity Recognition (NER Sequence Labeling vs. POS) [UPLOADED PYQ]

Topic 48: Rule-Based Machine Translation (Direct, Transfer & Interlingua)

Topic 50: Neural Machine Translation (End-to-End Transformer Seq2Seq)

Topic 52: Bias, Fairness & Stereotyping in Large Language Models
- Topic 45:** Sentiment Analysis (Document, Sentence & Aspect-Based)

Topic 47: Machine Translation Overview & The Vauquois Triangle

Topic 49: Statistical Machine Translation (SMT Noisy Channel Formulation)

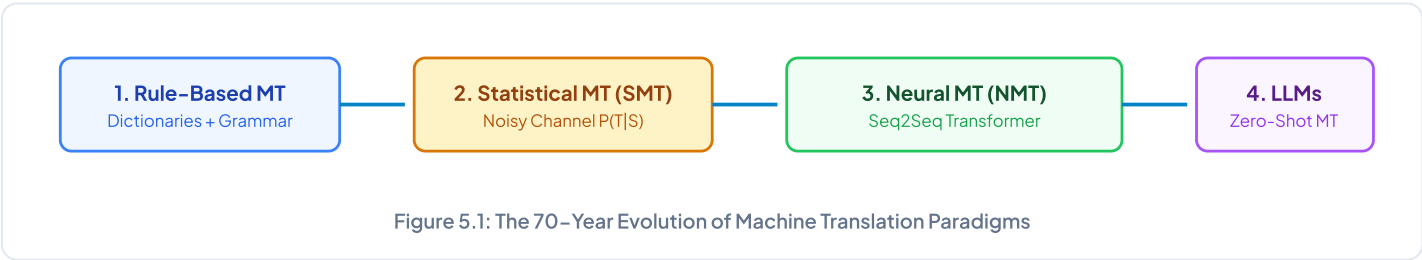
Topic 51: Chatbots & Conversational Dialogue Systems (NLU-DM-NLG Pipeline)

Topic 53: Model Explainability & Interpretability (Attention Heatmaps & LIME)

Topic 44 – 46: Text Classification, Sentiment Analysis & NER [UPLOADED PYQ]

Feature	Part-of-Speech (POS) Tagging	Named Entity Recognition (NER) [UPLOADED PYQ – CS633]
Objective	Assigns syntactic grammatical word classes ('NN', 'VB', 'JJ', 'DT').	Detects and classifies real-world semantic entity mentions.
Granularity	Every single token in the sentence receives a tag.	Only specific entity spans receive labels ('PERSON', 'ORG', 'LOC', 'DATE'). Non-entities are tagged 'O'.
Example	"Sundar [NNP] Pichai [NNP] leads [VBZ] Google [NNP]"	"Sundar Pichai [B-PER, I-PER] leads Google [B-ORG]" (BIO tagging scheme).

Topic 47 – 50: Machine Translation (MT) Paradigm Evolution



Statistical Machine Translation (Noisy Channel Model):

$$\hat{T} = \arg \max_T P(T | S) = \arg \max_T \left[\underbrace{P(S | T)}_{\text{Translation Model (Fidelity)}} \times \underbrace{P(T)}_{\text{Language Model (Fluency)}} \right]$$

Topic 51: Chatbots & Conversational Dialogue Architectures

The 3-Tier Task-Oriented Dialogue System Architecture

1. **Natural Language Understanding (NLU)**: Converts user utterance into structured semantic frame: Intent Classification (e.g., `BookFlight`) + Slot Filling (e.g., `Origin: "Ranchi"`, `Destination: "Delhi"`).
2. **Dialogue State Tracker (DST) & Policy Manager**: Maintains conversation history state and selects the next system action (e.g., `AskDepartureDate` or `ExecuteBookingAPI`).
3. **Natural Language Generation (NLG)**: Synthesizes natural, fluent response text from the structured action template.

Topic 52 & 53: Ethics, Bias Mitigation & Model Explainability

Ethical / Interpretability Challenge	Manifestation in Language Models	Engineering Mitigation Method
1. Dataset & Representation Bias	Stereotypical associations in word embeddings (e.g., "doctor" - "man" + "woman" $\approx$ "nurse").	Counterfactual data augmentation, debiasing projection subspaces.
2. Model Hallucination	Generating plausibly sounding but factually incorrect assertions with high confidence.	Retrieval-Augmented Generation (RAG), RLHF with truthfulness reward models.
3. Black-Box Interpretability	Complex billions-parameter Transformer decisions cannot be audited for high-stakes healthcare/law.	LIME (Local Interpretable Model-agnostic Explanations), Integrated Gradients, Cross-Attention Heatmaps.

M5 Active Recall & Exam Questions [UPLOADED PYQ]

Q1. [UPLOADED PYQ] Compare Rule-Based, Statistical, and Neural Machine Translation systems across 4 dimensions. (8 Marks)

1. **Knowledge Representation**: RBMT relies on explicit handwritten grammar rules and bilingual dictionaries; SMT uses phrase-based translation tables; NMT uses continuous distributed vector representations in a unified neural network.
2. **Fluency & Context**: RBMT produces robotic, literal translations; SMT handles local phrases well but struggles with long-distance reordering; NMT generates highly fluent translations with global sentence context.
3. **Development Cost**: RBMT requires decades of expert linguistic labor; SMT/NMT learn automatically from parallel sentence corpora.
4. **Hardware Requirements**: RBMT runs with low compute; NMT requires massive GPU accelerators for Transformer matrix multiplications.